

Theory of Computation

BSc Computer Science - Durham University

Matthew Emerson

October 2026

Contents

1	Module Introduction	1
1.1	Assessment	1
1.2	Timetable	1
2	Models of Computation	1
2.1	Finite-State Automata and Regular Languages	1
2.2	Büchi Automata and Omega-Regular Languages	3
2.3	Linear-Time Logic and Translating Into Büchi Automata	3
3	Algorithms and Complexity	3
3.1	Asymptotic Complexity and Notation	3
3.2	Greedy Algorithms	3
3.3	Divide and Conquer Algorithms	3
3.4	Dynamic Programming Algorithms	3
3.5	Machinings in Graphs	3
I	Appendix	4
4	Bibliography	4
4.1	Module Professors	4
4.2	References	4
4.3	List of Figures	4
4.4	Recommended Reading	4

1 Module Introduction

Theory of Computation has the module code COMP2181 and is led by Dr Barnaby Martin.

1.1 Assessment

This module is assessed through two summative benchtests, and one final exams, these can be seen in Table 1. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Benchtest 1	Week 11	-	2nd Feb	Dr Dantchev and Dr Mertzios	12.5%
Benchtest 2	Week 20	-	-	Dr Dantchev and Dr Mertzios	12.5%
Final Exam	May Exam	-	-	Dr Barnaby Martin	75%

Table 1: Assessments for Theory of Computation.

1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Tuesday	10:00 - 11:00	Lecture	CLC202	Dr Barnaby Martin
Wednesday	09:00 - 11:00	Problem Class	TLC116	-
Thursday	14:00 - 15:00	Lecture	CLC202	Dr Sukanya Pandey

Table 2: Michaelmas Term Timetable for Theory of Computation.

2 Models of Computation

This section is taught by Dr Barnaby Martin. [Check for recommended reading.](#)

2.1 Finite-State Automata and Regular Languages

2.1.1 Formal Definitions

Definition 2.1.1.1: A deterministic finite-state automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q is the finite set of states, Σ is a finite alphabet, $\delta : Q \times \Sigma \rightarrow Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

Definition 2.1.1.2: A DFA M recognises the language L if $L = \{w \mid M \text{ accepts } w\}$.

Definition 2.1.1.3: A language is called a regular language if some DFA recognises it.

Definition 2.1.1.4: A regular expression R defines a language $L(R)$. We shall eventually prove that $RE \equiv DFA$. R is a regular expression over the alphabet Σ if R is:

1. a for some $a \in \Sigma$.
2. ϵ (the empty string).
3. $\emptyset$ (the empty set).
4. $(R_1 \cup R_2)$ where R_1 and R_2 are regular expressions.
5. $R_1 \cdot R_2$ where R_1 and R_2 are regular expressions.
6. R_1^* where R_1 is a regular expression.

The list of regular operations can be seen in Table 3 where the operations labelled with $+$ are language theory specific.

Operation Name	Notation	Set Equivalent
Union	$A \cup B$	$\{x x \in A \text{ or } x \in B\}$
Intersection	$A \cap B$	$\{x x \in A \text{ and } x \in B\}$
Difference	$A \setminus B$	$\{x x \in A \text{ and } x \notin B\}$
Complement	$\overline{A}$	$\Sigma^* \setminus A$
Concatenation ⁺	$A \circ B$	$\{xy x \in A \text{ and } y \in B\}$
Star ⁺	A^*	$\{x_1x_2\dots x_k k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\} \equiv \cup_{i=0}^{\infty} A^i$

Table 3: Regular Operations.

2.1.2 Combining Automata

Sometimes we may want to implement an operation on multiple languages. We can do this by combining their automata. If we have that L_1 is recognised by $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and L_2 is recognised by $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$; we want to combine M_1 and M_2 into a new automaton M that would recognise $L_1 \cup L_2$.

You would think that we could just run the automata sequentially, and accept if either M_1 or M_2 accept, but this does not work. This is because after M_1 has run on the input, the input is exhausted and there is no way to rewind it in order to run M_2 .

Instead, we can attempt to run M_1 and M_2 on the input in parallel. We do this by using pairs of states (one for each DFA). Formally, the components of the new automaton $M = (Q, \Sigma, \delta, q_0, F)$ are:

1. The set of possible states contains every possible cross-set pair of Q_1 and Q_2 .

$$Q = Q_1 \times Q_2$$

2. For every $s \in Q_1, u \in Q_2, a \in \Sigma$. The transition function contains every transition from a pair of states from Q being given any input from Σ .

$$\delta((s, u), a) = (\delta_1(s, a), \delta_2(u, a))$$

3. The start state is the pair containing both start states from M_1 and M_2 .

$$q_0 = (q_1, q_2)$$

4. The set of accept states is every possible cross-set pair of accept states from F_1 and F_2 .

$$F = \{(s, u) \in Q | s \in F_1 \text{ or } u \in F_2\}$$

2.1.3 Non-Deterministic Finite Automaton

If, instead you wish to work with concatenation, such that $w \in L_1 \circ L_2$ only if there are words x and y such that $w = xy$, $x \in L_1$ and $y \in L_2$. You need to run M_1 on a prefix of w and then run M_2 on the rest. This must be done non-deterministically and stops becoming a DFA as all accept states of M_1 have to map to the start state of M_2 .

Definition 2.1.3.1: A non-deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is the finite set of states, Σ is a finite alphabet, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accept states.

Computing NFA

Before computing an NFA, an important convention to note is that a single step in a computation consists of first reading the next input symbol followed by taking any number of ϵ -transitions. This means that empty strings ϵ s can be freely added inside the actual word $\in \Sigma^*$, e.g., 010110 can be viewed as, say, $\epsilon 01\epsilon\epsilon 01\epsilon 01$.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA and $w = w_1w_2\dots w_n$ be a word over $\Sigma \cup \{\epsilon\}$. M accepts w if there exists a sequence of states $r_0, r_1, r_2, \dots, r_n$ satisfying the following conditions:

1. $r_0 = q_0$.
2. $r_{i+1} \in \delta(r_i, w_{i+1})$ for every $i, 0 \leq i \leq n - 1$.
3. $r_n \in F$.

2.1.4 Converting NFA into DFA

The difference between a DFA and NFA is that a DFA has a unique computation (path) and either accepts or rejects, while a NFA generally has many computation paths and accepts if at least one computation accepts and only rejects if all the computations reject.

Reached slide 22/55 (powerpoint says slide 21).

2.2 Büchi Automata and Omega-Regular Languages

2.3 Linear-Time Logic and Translating Into Büchi Automata

3 Algorithms and Complexity

This section is taught by Dr Sukanya Pandey. The recommended reading for this topic is:

- Introduction to Algorithms (Cormen T.H., et al., 2009).

3.1 Asymptotic Complexity and Notation

3.2 Greedy Algorithms

3.3 Divide and Conquer Algorithms

3.4 Dynamic Programming Algorithms

3.5 Machinings in Graphs

Part I

Appendix

4 Bibliography

4.1 Module Professors

Module Lead: Doctor Barnaby Martin

- Focus: Complexity theory, finite model theory, and the links between logic and complexity.
- Email: barnaby.d.martin@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

Lecturer: Doctor Sukanya Pandey

- Focus: Graph theory.
- Email: sukanya.pandey@durham.ac.uk
- Office Hours: Flexible, contact to arrange.

Lecturer: Doctor Stefan Dantchev

- Focus: Combinatorics.
- Email: s.s.dantchev@durham.ac.uk
- Office Hours: **To be confirmed.**

Lecturer: Doctor George Mertzios

- Focus: Algorithms and networks.
- Email: george.mertzios@durham.ac.uk
- Office Hours: **To be confirmed.**

4.2 References

4.3 List of Figures

4.4 Recommended Reading

4.4.1 Algorithms and Complexity

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). Introduction to Algorithms. 3rd edn. MIT Press.